

Lab Report No 5
Procedure and Stacks
Microprocessor and Interfacing techniques



Submitted By:

Huzaifa Shahbaz

21-CE-009

Submitted to:

Engr. Bushra Fiaz

Dated:

Week 05

Department of Computer Engineering,
HITEC University, Taxila

Lab Task No 1:

Solution:

Brief description (3-5 lines)

Write a program in which all general purpose registers are utilized. During program call a procedure in which again all four general purpose registers are used. But before jumping to procedure save all registers contents by using stack operations and after returning from stack, retain the registers values. Use simple arithmetic operations.

The code

```
org 100h
section .data
section .text
start:
    mov ax, 1234
    mov bx, 5678
    mov cx, 1111
    mov dx, 9999
    push ax
    push bx
    push cx
    push dx
    call myProcedure
    pop dx
    pop cx
    pop bx
    pop ax
    mov ah, 4Ch
    int 21h
myProcedure:
    push ax
    push bx
    push cx
    push dx
    mov ax, [bp+6]
    mov bx, [bp+8]
    add cx, ax
    add cx, bx
    pop dx
    pop cx
    pop bx
    pop ax
    ret
```

The results (Screenshot)

registers		F400:0200	F400:0204
AX	4C D2	P4200: FF 255 RES	B106 D1
BX	16 2E	P4201: FF 255 RES	INT 021h
CX	04 57	P4202: CD 205 =	INT
DX	27 0F	P4203: 21 013 T	ADD EBX + SI, AL
CS	F400	P4204: 00 000 NULL	ADD EBX + SI, AL
IP	02 04	P4205: 00 000 NULL	ADD EBX + SI, AL
SS	07 00	P4206: 00 000 NULL	ADD EBX + SI, AL
SP	FFF8	P4207: 00 000 NULL	ADD EBX + SI, AL
BP	0000	P4208: 00 000 NULL	ADD EBX + SI, AL
SI	0000	P4209: 00 000 NULL	ADD EBX + SI, AL
DI	0000	P420A: 00 000 NULL	ADD EBX + SI, AL
DS	07 00	P420B: 00 000 NULL	ADD EBX + SI, AL
ES	07 00	P420C: 00 000 NULL	ADD EBX + SI, AL
		P420D: 00 000 NULL	...
		P420E: 00 000 NULL	
		P420F: 00 000 NULL	
		P4210: 00 000 NULL	
		P4211: 00 000 NULL	
		P4212: 00 000 NULL	
		P4213: 00 000 NULL	
		P4214: 00 000 NULL	
		P4215: 00 000 NULL	
		P4216: 00 000 NULL	
		P4217: 00 000 NULL	
		P4218: 00 000 NULL	
		P4219: 00 000 NULL	
		P421A: 00 000 NULL	
		P421B: 00 000 NULL	
		P421C: 00 000 NULL	

Lab Task No 2:

Solution:

Brief description (3-5 lines)

Write a program that virtually create three different stacks. With SP= 1000; SP=2000; SP=3000 Push first 10 odd numbers to the first stack, first ten even numbers to the second stack and write all ones to the last stack.

The code

```

01 ORG 100h
02 section .data
03     odd_numbers db 1, 3, 5, 7, 9, 11, 13, 15, 17, 19
04     even_numbers db 2, 4, 6, 8, 10, 12, 14, 16, 18, 20
05     ones db 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
06 section .text
07     global _start
08 _start:
09     mov ax, 1000
10     mov ss, ax
11     mov sp, ax
12     mov cx, 10
13     mov si, 0
14 push_odd_numbers:
15     push word [odd_numbers + si]
16     add si, 2
17     loop push_odd_numbers
18     mov ax, 2000
19     mov ss, ax
20     mov sp, ax
21     mov cx, 10
22     mov si, 0
23 push_even_numbers:
24     push word [even_numbers + si]
25     add si, 2
26     loop push_even_numbers
27     mov ax, 3000
28     mov ss, ax
29     mov sp, ax
30     mov cx, 10
31     mov si, 0
32 push_ones:
33     push word [ones + si]
34     add si, 2
35     loop push_ones
36     int 20h
37     ret

```

The results (Screenshot)

registers		0700:0100		0700:0100	
	H L				
AX	09 07	07100: 01 001 0	ADD AL, [SI]		
BX	00 00	07101: 03 003 0	PUSH ES		
CX	20 EF	07102: 05 005 0	OR BP, SI, CL		
DX	00 00	07103: 07 007 0	OR AL, 00h		
		07104: 09 009 0	ADC BP, SI, DL		
		07105: 0B 011 0	ADC AL, 01h		
		07106: 0D 013 0	ADD BX, DI, AX		
		07107: 0F 015 0	ADD BX, DI, AX		
		07108: 11 017 0	ADD BX, DI, AX		
		07109: 13 019 0	ADD BX, DI, AX		
IP	0100	0710A: 15 01A 0	ADD BX, SI + 003Eh, DI		
SS	0700	0710B: 17 01B 0	MOV SS, AX		
SP	FFFF	0710C: 19 01C 0	MOV SP, AX		
BP	0000	0710D: 1B 01D 0	MOV CX, 0000h		
		0710E: 1D 01E 0	MOV SI, 0000h		
		0710F: 1F 01F 0	PUSH w, [SI] + 0010h		
		07110: 21 021 0	ADD SI, 02h		
		07111: 23 023 0	LOOP 012h		
		07112: 25 025 0	MOV BX, 0070h		
		07113: 27 027 0	MOV SS, AX		
		07114: 29 029 0	MOV SP, AX		
		07115: 2B 02B 0	MOV CX, 0000h		
		07116: 2D 02D 0	MOV SI, 0000h		
		07117: 2F 02F 0	PUSH w, [SI] + 0010h		
		07118: 31 031 0	ADD SI, 02h		
		07119: 33 033 0	LOOP 014h		
		0711A: 35 035 0	MOV AX, 0000h		

Lab Task No 3:

Solution:

Brief description (3-5 lines)

Create two stacks, swap the data of both the stacks.

The code

```

01  org 100h
02  .model small
03  .stack 100h
04  .data
05  stack1 db 10 dup(0)
06  stack2 db 10 dup(0)
07  temp db ?
08  .code
09  main proc
10      mov ax, @data
11      mov ds, ax
12      mov ax, 1
13      mov bx, 2
14      push ax
15      push bx
16      mov ax, 3
17      mov bx, 4
18      push ax
19      push bx
20      pop ax
21      pop bx
22      push bx
23      push ax
24      mov cx, 2
25      mov si, 0
26      mov ah, 2
27      mov dl, ' '
28  print_stack1:
29      mov al, stack1[si]
30      add al, 48
31      int 21h
32      inc si
33      loop print_stack1
34      mov cx, 2
35      mov si, 0
36      mov ah, 2
37      mov dl, ' '
38  print_stack2:
39      mov al, stack2[si]
40      add al, 48
41      int 21h
42      inc si
43      loop print_stack2
44      mov ah, 4ch
45      int 21h
46  main endp
47  end main
48  ret

```

The results (Screenshot)

registers			F400:0204		F400:0204	
	H	L				
AX	4C	20	F4200: FF 255 RES		BIOS DI	
BX	00	03	F4201: FF 255 RES		INT 021h	
CX	00	00	F4202: CD 205 =		034	
DX	00	20	F4203: 21 033 !		ADD EBX + SI, AL	
CS	F400		F4204: CF 207 !		ADD EBX + SI, AL	
IP	0204		F4205: 00 000 NULL		ADD EBX + SI, AL	
SS	0710		F4206: 00 000 NULL		ADD EBX + SI, AL	
SP	00F2		F4207: 00 000 NULL		ADD EBX + SI, AL	
BP	0000		F4208: 00 000 NULL		ADD EBX + SI, AL	
SI	0002		F4209: 00 000 NULL		ADD EBX + SI, AL	
DI	0000		F420A: 00 000 NULL		ADD EBX + SI, AL	
DS	0720		F420B: 00 000 NULL		ADD EBX + SI, AL	
ES	0700		F420C: 00 000 NULL		ADD EBX + SI, AL	
			F420D: 00 000 NULL		ADD EBX + SI, AL	
			F420E: 00 000 NULL		ADD EBX + SI, AL	
			F420F: 00 000 NULL		ADD EBX + SI, AL	
			F4210: 00 000 NULL		ADD EBX + SI, AL	
			F4211: 00 000 NULL		ADD EBX + SI, AL	
			F4212: 00 000 NULL		ADD EBX + SI, AL	
			F4213: 00 000 NULL		ADD EBX + SI, AL	
			F4214: 00 000 NULL		ADD EBX + SI, AL	
			F4215: 00 000 NULL		ADD EBX + SI, AL	
			F4216: 00 000 NULL		ADD EBX + SI, AL	
			F4217: 00 000 NULL		ADD EBX + SI, AL	
			F4218: 00 000 NULL		ADD EBX + SI, AL	

Lab Task No 4:

Solution:

Brief description (3-5 lines)

Write a program to reverse the stored string using stack.

The code

```

01 org 100h
02 .model small
03 .stack 100h
04 .data
05 string db 'hello world$'
06 len equ $-string
07 stack db len dup(0)
08 top dw 0
09 .code
10 main proc
11     mov ax, @data
12     mov ds, ax
13     mov cx, len
14     mov si, 0
15 push_loop:
16     mov al, string[si]
17     push ax
18     inc si
19     loop push_loop
20     mov cx, len
21     mov si, 0
22 pop_loop:
23     pop ax
24     mov stack[si], al
25     inc si
26     loop pop_loop
27     mov dx, offset stack
28     mov ah, 9
29     int 21h
30     mov ah, 4ch
31     int 21h
32 main endp
33 end main
34 ret

```

The results (Screenshot)

registers		F400:0204		F400:0204	
	H L				
AX	09 68	F4200: FF 255 RES		BIOS D1	
BX	00 00	F4201: FF 255 RES		INT 021h	
CX	00 00	F4202: CD 205 =		IRET	
DX	07 10	F4203: 21 033 +		ADD EBX + SI, AL	
CS	F400	F4204: 00 000		ADD EBX + SI, AL	
IP	0204	F4205: 00 000 NULL		ADD EBX + SI, AL	
SS	0710	F4206: 00 000 NULL		ADD EBX + SI, AL	
SP	00FA	F4207: 00 000 NULL		ADD EBX + SI, AL	
BP	0000	F4208: 00 000 NULL		ADD EBX + SI, AL	
SI	000C	F4209: 00 000 NULL		ADD EBX + SI, AL	
DI	0000	F420A: 00 000 NULL		ADD EBX + SI, AL	
DS	0720	F420B: 00 000 NULL		ADD EBX + SI, AL	
ES	0700	F420C: 00 000 NULL		ADD EBX + SI, AL	
		F420D: 00 000 NULL		ADD EBX + SI, AL	
		F420E: 00 000 NULL		ADD EBX + SI, AL	
		F420F: 00 000 NULL		ADD EBX + SI, AL	
		F4210: 00 000 NULL		ADD EBX + SI, AL	
		F4211: 00 000 NULL		ADD EBX + SI, AL	
		F4212: 00 000 NULL		ADD EBX + SI, AL	
		F4213: 00 000 NULL		ADD EBX + SI, AL	
		F4214: 00 000 NULL		ADD EBX + SI, AL	
		F4215: 00 000 NULL		ADD EBX + SI, AL	
		F4216: 00 000 NULL		ADD EBX + SI, AL	
		F4217: 00 000 NULL		ADD EBX + SI, AL	
		F4218: 00 000 NULL		ADD EBX + SI, AL	
		F4219: 00 000 NULL		ADD EBX + SI, AL	
		F421A: 00 000 NULL		ADD EBX + SI, AL	